

Zoho CRM Integration

1. Overview

1.1 Description

Zoho CRM Integration connects **Zoho CRM** (cloud-based CRM platform) with **Newo Platform**.

It enables:

- Checking real-time appointment availability via Zoho Events
- Booking appointments with automatic contact creation in Zoho CRM
- Cancelling existing appointments (event deletion)
- Automatic deal creation and pipeline stage management based on conversation outcomes
- Lead creation and conversation note tracking for client history
- Contact lookup and deduplication by phone number
- OAuth 2.0 authentication with multi-region support (.com, .eu, .in, .au)

This integration is designed for **sales teams, service providers, and business owners** who need **automated appointment scheduling and CRM synchronization through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a client asks about available slots → Query Zoho Events calendar and return free time slots
- When a client wants to book an appointment → Find or create contact, then create calendar event in Zoho CRM
- When a client wants to cancel an appointment → Delete the event from Zoho CRM
- When a conversation ends → Create/update CRM deal with AI-selected pipeline stage
- When a new caller contacts → Create lead in Zoho CRM and attach conversation notes
- When a returning caller contacts → Find existing contact and add conversation summary as note

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✓	Via Zoho Events calendar search
Book Appointment	✓	Creates Zoho Event with contact as participant
Cancel Appointment	✓	Deletes event from Zoho CRM
Contact Lookup	✓	By phone number with auto-normalization
Contact Creation	✓	Automatic during booking if not found
Deal Creation	✓	With pipeline stage and contact link
Deal Stage Update	✓	AI-powered stage selection via LLM (Gemini)
Lead Creation	✓	For new callers on session end
Conversation Notes	✓	Summary + transcript attached to contact
Multi-Region Support	✓	.com, .eu, .in, .au Zoho datacenters

OAuth 2.0 Token Management	✓	Auto-refresh with retry on 401
Silent Availability Preload	✓	Optional check on conversation start
Configurable Slot Duration	✓	Default 30 min
Multi-Location Support	✗	Single calendar/user per instance
Reschedule Appointment	✗	Cancel + rebook as workaround

2. Requirements

Before installation:

- Active **Zoho CRM** account
- **OAuth 2.0 authorization** via Newo's registered app
- Zoho account **region** (.com, .eu, .in, or .au)

3. How to Setup

3.1 Step 1 — Authorize in Zoho CRM

1. In Newo platform, set `zoho_region` to your Zoho datacenter region
2. Click the Zoho authorization link provided by the platform
3. Authorize the Newo app in your Zoho CRM account
4. Grant the requested scopes (CRM org, settings, users, modules, etc.)
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `zoho_oauth_code` attribute

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
zoho_oauth_code	✓	One-time OAuth authorization code
zoho_region	✓	Zoho datacenter: .com, .eu, .in, .au (default: .com)
zoho_timezone	✗	Appointment timezone (default: project timezone)
zoho_duration	✗	Slot duration in minutes (default: 30)
zoho_availability_days	✗	Days ahead for availability search
zoho_enable_slot_check	✗	Enable availability checking (default: True)
zoho_enable_booking	✗	Enable booking capability (default: True)
zoho_enable_cancellation	✗	Enable cancellation capability (default: True)

zoho_update_pipeline	✗	Enable auto deal stage updates (default: False)
zoho_pipeline_criteria	✗	Custom business rules for AI stage selection
zoho_check_existing_client	✗	Verify client exists before booking (default: True)
zoho_check_availability_on_conversation_start	✗	Auto-check slots on session start (default: False)

3. Click **Save + Publish All**

4. On publish, the SetupFlow automatically:

- Exchanges the OAuth code for access and refresh tokens
- Fetches Zoho CRM users and auto-selects the owner
- Fetches deal field definitions and pipeline stages
- Injects booking and availability payload schemas for the agent

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration combines **scheduling** (Events) and **CRM** (Contacts, Deals, Leads, Notes) capabilities via the event-driven system.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Zoho CRM

Available Triggers

Trigger	Event	Description
Check Availability	get_available_slots	When the agent needs to look up open slots
Book Appointment	book_slot	When the agent confirms a booking
Cancel Appointment	convo_agent_cancel_booking	When the agent processes a cancellation
Session Ended	session_ended	Triggers client history (lead/notes) + pipeline update
Contact Request	zoho_goc_contact_request	Internal: get or create contact
Deal Request	modify_deal_request	Internal: create or update deal
Session Start	session_started	Optional silent availability preload
Setup	project_publish_finish	Initializes integration on deploy

Available Actions

- **Get Or Create Contact** — Find contact by phone or create new (GET/POST `/Contacts`)

- **Create Event** — Book appointment as calendar event (POST /Events)
- **Delete Event** — Cancel appointment (DELETE /Events/{id})
- **Search Events** — Check availability by date range (GET /Events/search)
- **Create Deal** — Create CRM deal with pipeline stage (POST /Deals)
- **Update Deal** — Change deal stage (PUT /Deals/{id})
- **Create Lead** — Create lead for new callers (POST /Leads)
- **Add Note** — Attach conversation summary to contact (POST /Contacts/{id}/Notes)
- **Get Users** — Discover Zoho CRM users (GET /users)
- **Get Deal Stages** — Fetch pipeline field definitions (GET /settings/fields?module=Deals)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Zoho Integration
    participant T as Zoho OAuth
    participant API as Zoho CRM API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: POST /oauth/v2/token (code)
    T-->>I: access_token + refresh_token
    I->>API: GET /users
    API-->>I: Users list
    I->>API: GET /settings/fields?module=Deals
    API-->>I: Deal stages
    I->>A: Schemas injected, tools configured

    Note over U,API: Availability Check
    U->>A: "Any slots this week?"
    A->>I: get_available_slots (date)
    I->>API: GET /Events/search (date range)
    API-->>I: Existing events
    I->>I: Calculate free slots
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:30 PM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: GET /Contacts/search?phone=(phone)
    alt Contact not found
        I->>API: POST /Contacts (name, phone, email)
    end
    API-->>I: contact_id
    I->>API: POST /Events (dateTime, participant: contact_id)
    API-->>I: event_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"
```

```

Note over U,API: Cancellation
U->>A: "Cancel my appointment"
A->>I: convo_agent_cancel_booking (booking_id)
I->>API: DELETE /Events/(booking_id)
API-->>I: Confirmation
I->>A: result_success (cancelled)
A->>U: "Your appointment has been cancelled"

Note over U,API: Post-Conversation (session_ended)
A->>I: session_ended (summary, transcript, phone)
I->>API: GET /Contacts/search?phone=(phone)
alt Contact found
    I->>API: POST /Contacts/(id)/Notes (summary, transcript)
else New caller
    I->>API: POST /Leads (name, phone, email)
end
I->>I: LLM analyze conversation -> select deal stage
I->>API: POST or PUT /Deals (stage, contact)
API-->>I: deal_id

```

AvailabilityFlow

```

flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE["Slot check<br>enabled?"]
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| RANGE["Calculate date range:<br>date → date + availability_days"]
    RANGE --> API["GET /Events/search<br>{Start_DateTime between<br>start and end}"]
    API --> OK["HTTP 200?"]
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| CALC["Parse existing events<br>Calculate free slots<br>(duration-based)"]
    CALC --> TZ["Convert to<br>business timezone"]
    TZ --> OUT["result_success<br>{availability[]}"]

```

BookingFlow

```

flowchart TD
    E["book_slot"] --> GATE["Booking<br>enabled?"]
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| EX["Extract customerInfo:<br>name, phone, email"]
    EX --> GOC["GetOrCreateContact<br>{phone, name, email}"]
    GOC --> SEARCH["GET /Contacts/search<br>?phone={phone}"]
    SEARCH --> FOUND["Contact<br>found?"]
    FOUND -->|"Yes"| CID["Use contact_id"]
    FOUND -->|"No"| CREATE["POST /Contacts<br>{name, phone, email}"]
    CREATE --> CID
    CID --> UTC["Convert time<br>to UTC"]

```

```

UTC --> EVENT["POST /Events<br>{Start_DateTime, End_DateTime,<br>Event_Title,
Participants}"]
EVENT --> OK{"HTTP 200?"}
OK -->|"No"| ERR["ResultError"]
OK -->|"Yes"| OUT["result_success<br>{booking_id}"]

```

CancellationFlow

```

flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| API["DELETE /Events/{booking_id}"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]

```

PipelineFlow (Post-Conversation)

```

flowchart TD
    E["zoho_session_ended"] --> GATE{"Pipeline update<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| LLM["LLM analyze conversation:<br>summary, transcript,
<br>success, termination reason"]
    LLM --> STAGE["Extract stage_id,<br>client name, explanation"]
    STAGE --> DEAL["Search existing deal<br>GET /Deals/search<br>{owner_id,
contact_id}"]
    DEAL --> FOUND{"Deal<br>found?"}
    FOUND -->|"Yes"| UPDATE["PUT /Deals/{id}<br>{status_id}"]
    FOUND -->|"No"| CREATE["POST /Deals<br>{name, stage, contact}"]
    UPDATE --> DONE1["Done"]
    CREATE --> DONE2["Done"]

```

ClientHistoryFlow (Post-Conversation)

```

flowchart TD
    E["session_ended"] --> SEARCH["GET /Contacts/search<br>?phone={phone}"]
    SEARCH --> FOUND1{"Contact<br>found?"}
    FOUND1 -->|"No"| LEAD["POST /Leads<br>{name, phone, email}"]
    FOUND1 -->|"Yes"| NOTE["POST /Contacts/{id}/Notes<br>{summary, transcript,
<br>category, session type}"]
    NOTE --> PIPELINE["Trigger PipelineFlow<br>via zoho_session_ended"]
    LEAD --> DONE1["Done"]
    PIPELINE --> DONE2["Done"]

```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel) by interacting with the agent. The GetOrCreateContactFlow also has a dedicated webhook test endpoint (`goc_wh`).

4.4 Testing and Verification

To test the integration:

1. **Setup:** Publish the project and verify logs show successful OAuth token exchange, user fetch, and deal stages fetch
2. **Availability:** Ask the agent "What slots are available this week?" — verify free slots calculated from Zoho Events calendar
3. **Booking:** Complete a booking conversation — verify:
 - Contact created/found in Zoho CRM
 - Calendar event created with correct time and participant
4. **Cancellation:** Cancel an appointment — verify event deleted from Zoho CRM
5. **Client History:** After a conversation ends, verify:
 - Contact note added with summary and transcript
 - Lead created if contact didn't exist
6. **Pipeline:** If `zoho_update_pipeline` is enabled, verify deal stage updated based on conversation outcome

If no action occurs:

- Ensure OAuth tokens were obtained (check `zoho_access_token` is not empty)
- Verify `zoho_region` matches your Zoho datacenter
- Confirm the OAuth code hasn't expired (codes are single-use)
- Check that feature flags are enabled (`zoho_enable_booking` , `zoho_enable_slot_check` , `zoho_enable_cancellation`)
- Verify `zoho_user_id` was populated during setup
- Re-publish the project if credentials were changed

Note: This integration performs real operations in Zoho CRM. Contacts, events, deals, leads, and notes created through the agent are reflected in the live CRM system.

5. FAQ

Q: Does the integration import historical CRM data? A: No. Only appointments and records created after activation are managed. Client history flow creates notes for new conversations only.

Q: Can I connect multiple Zoho CRM accounts? A: Each integration instance supports one Zoho account and user. For multiple accounts, create separate integration instances.

Q: Which Zoho regions are supported? A: All four Zoho datacenters: `.com` (US), `.eu` (Europe), `.in` (India), `.au` (Australia). Set the `zoho_region` attribute to match your account's datacenter.

Q: How does the AI select the deal pipeline stage? A: After each conversation, an LLM (Gemini) analyzes the conversation summary, outcome success, detailed steps, and transcript. It applies custom business rules from `zoho_pipeline_criteria` and selects the most appropriate deal stage. The decision includes an explanation.

Q: What happens if a contact already exists in Zoho? A: The integration searches contacts by phone number. If found, the existing contact is used for booking and notes are appended. If not found, a new contact is created automatically.

Q: How does availability checking work? A: The integration queries Zoho Events within a date range and calculates free time slots by finding gaps between existing events. Slot duration is configurable via `zoho_duration` (default: 30 minutes).

Q: What happens if the OAuth token expires? A: The integration automatically detects HTTP 401 responses and refreshes the token using the refresh token. The new access token is persisted and the failed request is retried.

Q: What CRM records are created during a conversation? A: Depending on configuration: **Contact** (during booking), **Event** (appointment), **Deal** (with pipeline stage), **Lead** (for new callers), and **Note** (conversation summary attached to contact). Not all are created every time — it depends on which flows are enabled.